Planet Wars Competition Entry
Planet Warriors MCTS Agent V2

Monte Carlo Tree Search Agent for fully observable Planet Wars

Planet Warriors: Melih Ustun, Ben Pittman, Nikolay Popov, Deniz Atar

Queen Mary University of London

Agent Overview

- Goal: choose the best action by simulating future game states and evaluating the final position.
- Builds a partial game tree from the current game state at every tick.
- Explores possible actions from owned source planets to enemy or neutral targets and evaluates how well they perform in future.
- Uses UCB1 formula to balance visiting known, high-value branches (exploitation) with unexplored branches (exploration).
- Returns the action leading to the child node with the most visits after performing as many iterations as possible within the given time limit.

System Design

1. Generate actions: each free owned source can target neutral or enemy planets. V1 - pure MCTS considering all possible source-target pairs –> V2 - prunes action space using heuristics to rule out bad actions.
2. Selection: traverse down the tree using UCB1 until reaching a node that hasn't been fully expanded (has untried actions).
3. Expansion: sample an untried action and step the forward model one tick.
4. Rollout: simulate random actions up to 200 moves in the future.
5. Evaluate the final position once a terminal state or maximum rollout depth is reached: ship difference + 20 x growth rate difference + planet count difference.
6. Backpropagation: add final rollout value to total node value and increment visit count along the path back to root.

Results

Benchmark test results (100 games per test):

- vs PureRandomAgent: 100%
- vs BetterRandomAgent: 100%
- vs CarefulRandomAgent: 100%
- vs GreedyHeuristic: 94%
- vs SimpleEvoAgent: 100%

Test config: 20 planets, 2000 ticks, growth rate: 0.05-0.2, transporter speed: 3.5

Context from implementation:

- Configurable time limit per decision, rollout depth up to 200 moves.
- Strong results, but far computationally heavier than simple heuristic agents.

Analysis and Insights

Pros

- Simulates moves ahead instead of relying on a fixed defend/expand/attack rule order.
- UCB1 naturally revisits promising actions while still exploring alternatives.
- Evaluation includes production and planet count, not only current ships.
- Pruning eliminates wasted time on useless/garbage actions.
- User-set time limit allows flexibility between performance and speed.

Cons

- Rollout policy is random, so estimates can be noisy under tight iteration budgets.
- Opponent model defaults to DoNothingAgent, limiting adversarial prediction.
- Search tree is reconstructed on each tick, wasting potential for a better search if the tree was to be cached and updated.
- Actions are limited to sending discrete fractions (½ or ¾) of a source's ships instead of any integer amount in order to reduce the search space, trading off flexibility and better possible actions in exchange for time-efficiency and tractability.

Future Work

- Cache the search tree to reuse information between ticks and reduce repeated computations.
- Replace random rollouts with a lightweight heuristic rollout policy.
- Further refine weights for differences in ships, growth rate, and planet ownership, consider normalising scores.
- Pair MCTS with neural networks for better move selection and evaluation, along with reinforcement learning/self play.
- Consider expanding the agent’s search space with a wider range of ships to send from each source, for better actions with fine-tuned fleets.